

✓ Lab 7 Pedro Martins

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

✓ Preliminary

✓ Packages to install

- ucimlrepo (for grabbing datasets)
- transformers
- autogen
- jupyter (if you aren't on the latest version, there's a dependency in tqdm that complains)

Install with pip

- pip install ucimlrepo transformers autogen jupyter

From lab 3

- pandas, numpy, matplotlib.pyplot, seaborn, tqdm, torch, sklearn

Install everything with pip

- pip install ucimlrepo transformers autogen pandas numpy matplotlib seaborn tqdm torch scikit-learn jupyter

If you need to install stuff on colab

```
!pip install ucimlrepo transformers autogen pandas numpy matplotlib seaborn tqdm torch scikit-learn jupyter
```



```
Requirement already satisfied: decorator in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipykernel->jupyter)
Requirement already satisfied: pickleshare in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipykernel->jupyter)
Requirement already satisfied: backcall in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipykernel->jupyter)
Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.11/dist-packages (from ipython>=7.23.1->ipykernel->jupyter)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.11/dist-packages (from jupyter-core->jupyterlab->jupyter-client)
Collecting jupyter-client>=6.1.12 (from ipykernel->jupyter)
  Downloading jupyter_client-7.4.9-py3-none-any.whl.metadata (8.5 kB)
Collecting jupyter-events>=0.11.0 (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter)
  Downloading jupyter_events-0.12.0-py3-none-any.whl.metadata (5.8 kB)
Collecting jupyter-server-terminals>=0.4.4 (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter)
  Downloading jupyter_server_terminals-0.5.3-py3-none-any.whl.metadata (5.6 kB)
Collecting overrides>=5.0 (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter)
  Downloading overrides-7.7.0-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: websocket-client>=1.7 in /usr/local/lib/python3.11/dist-packages (from jupyter-server<3,>=2.4.0->jupyterlab->jupyter)
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.11/dist-packages (from argon2-cffi->notebook->jupyterlab-server)
Requirement already satisfied: entrypoints in /usr/local/lib/python3.11/dist-packages (from jupyter-client>=6.1.12->ipykernel)
Requirement already satisfied: babel>=2.10 in /usr/local/lib/python3.11/dist-packages (from jupyterlab-server<3,>=2.27.1->jupyterlab)
```

Intro

The goal of this lab is to give you an idea of how you could use agents to help with physics tasks. It will also introduce you to AutoGen, one of the more popular frameworks at the moment for designing custom agentic workflows. We won't make use of all the tools it provides, just the very basics. Note also that many of the most interesting things one can do with AI-powered agents (see topics like retrieval augmented generation (RAG)) require very large models to be performant, and many techniques require continuous/repeated training. This means large resource requirements, so for this lab we will just be using a very small LLM (Llama: TinyLlama-1.1B-Chat-v1.0). The results from this are nowhere near as good as something like chatGPT, but it should give you an idea of how a more advanced model (or models) might be able to do something really helpful/cool. This is an area of current research, so it will be interesting to see what they can do!

Also because of the limited size of the model, the text parsing needs to be very mechanical, and in some places a bit obtuse. The better (and ideally specially trained for an agentic workflow, see RAG) your model is, the more this can be relaxed. If you're interested in working with agents in a more user-friendly way (hiding a lot of the mechanics that are on display here), check out sites like n8n or Google Gemini (be aware that these can require you to provide a lot of permissions). You can also feel free to replace TinyLlama with a call to a larger model using API keys if you have some.

Finally, we would like to emphasize the use of copilot/similar tools for this lab in particular. These are agents too! And they are definitely the most performant agents you have easy access to.

✓ Lab

- There are two datasets, "mnist" (from Lab 3) and "solar_flare"
- Try to get everything working with mnist, then try adding solar_flare
 - This is one way agents can be helpful, since they can analyze a dataset you've never seen before and take a first crack at it much faster than you can

✓ Imports

```
# Import necessary packages
```

```
%matplotlib inline
```

```
from autogen import ConversableAgent, AssistantAgent
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tqdm
import torch
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from ucimlrepo import fetch_ucirepo
```

✓ Lab 3 code (run and then minimize this)

- Mostly just lab 3 code packaged into functions

```
# Load MNIST Dataset in Numpy
def load_mnist_data():
    # 1000 training samples where each sample feature is a greyscale image with shape (28, 28)
    # 1000 training targets where each target is an integer indicating the true digit
    mnist_train_features = np.load('mnist_train_features.npy')
    mnist_train_targets = np.load('mnist_train_targets.npy')

    # 100 testing samples + targets
    mnist_test_features = np.load('mnist_test_features.npy')
    mnist_test_targets = np.load('mnist_test_targets.npy')

    # Print the dimensions of training sample features/targets
    #print(mnist_train_features.shape, mnist_train_targets.shape)
    # Print the dimensions of testing sample features/targets
    #print(mnist_test_features.shape, mnist_test_targets.shape)

    return mnist_train_features, mnist_train_targets, mnist_test_features, mnist_test_targets

def flatten_features(features):
    # Flatten the features from (28, 28) to (784,)
    return features.reshape(features.shape[0], -1)

def scale_features(features):
    scaler = StandardScaler()
    return scaler.fit_transform(features)

# More general function to load datasets, including solar_flare
def load_dataset(dataset_name: str = ""):
    if dataset_name == "mnist":
        train_features, train_targets, test_features, test_targets = load_mnist_data()
        train_features = flatten_features(train_features)
        test_features = flatten_features(test_features)
        train_features = scale_features(train_features)
        test_features = scale_features(test_features)

    elif dataset_name == "solar_flare":
        # Load the solar flare dataset
        solar_flare = fetch_ucirepo(id=89)

        # Simplifying slightly for the sake of this example
        solar_flare.data.targets = solar_flare.data.targets['severe flares']

        # Split the solar flare dataset into train and test sets (90:10 split)
        train_features, test_features, train_targets, test_targets = train_test_split(
            solar_flare.data.features, solar_flare.data.targets, test_size=0.1, random_state=42
        )

        # Onehot encode modified Zurich class, largest spot size, spot distribution
        onehot_columns = ["modified Zurich class", "largest spot size", "spot distribution"]
        for col in onehot_columns:
            onehot = pd.get_dummies(train_features[col], prefix=col)
            train_features = pd.concat([train_features, onehot], axis=1)
            train_features.drop(col, axis=1, inplace=True)

            onehot = pd.get_dummies(test_features[col], prefix=col)
            test_features = pd.concat([test_features, onehot], axis=1)
            test_features.drop(col, axis=1, inplace=True)

        # Scale the features
        train_features = scale_features(train_features)
        test_features = scale_features(test_features)

        # Convert targets to numpy arrays
        train_targets = train_targets.to_numpy()
        test_targets = test_targets.to_numpy()

    else:
        raise ValueError(f"Unknown dataset: {dataset_name}")

    # train-test split
    train_features, val_features, train_targets, val_targets = train_test_split(train_features, train_targets, test_size=0.2)

    return train_features, train_targets, val_features, val_targets, test_features, test_targets
```

```

# Train
def train_model(model, train_features, train_targets, validation_features, validation_targets,
                test_features=None, test_targets=None, learning_rate=0.0015, epochs=80, batch_size=64):
    """
    Train a neural network model on the provided data.

    Parameters:
        model: PyTorch model to train
        train_features: Training features as numpy array
        train_targets: Training targets as numpy array
        validation_features: Validation features as numpy array
        validation_targets: Validation targets as numpy array
        test_features: Test features as numpy array (optional)
        test_targets: Test targets as numpy array (optional)
        learning_rate: Learning rate for optimizer
        epochs: Number of training epochs
        batch_size: Batch size for training

    Returns:
        tuple: (trained model, training loss list, validation accuracy list)
    """
    # Initialize tracking lists
    train_loss_list = np.zeros(epochs)
    validation_accuracy_list = np.zeros(epochs)

    # Convert numpy arrays to PyTorch tensors
    train_inputs = torch.from_numpy(train_features).float()
    train_targets = torch.from_numpy(train_targets).long()

    validation_inputs = torch.from_numpy(validation_features).float()
    validation_targets = torch.from_numpy(validation_targets).long()

    if test_features is not None and test_targets is not None:
        test_inputs = torch.from_numpy(test_features).float()
        test_targets = torch.from_numpy(test_targets).long()
        test_dataset = TensorDataset(test_inputs, test_targets)
        test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    # Create dataloaders
    train_dataset = TensorDataset(train_inputs, train_targets)
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    validation_dataset = TensorDataset(validation_inputs, validation_targets)
    validation_loader = DataLoader(validation_dataset, batch_size=batch_size, shuffle=False)

    # Setup optimizer and scheduler
    loss_func = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

    # Move model to GPU if available
    if torch.cuda.is_available():
        model = model.cuda()
        train_inputs = train_inputs.cuda()
        validation_inputs = validation_inputs.cuda()
        validation_targets = validation_targets.cuda()

    # Training Loop
    for epoch in tqdm.trange(epochs):
        model.train() # Set model to training mode
        running_loss = 0.0

        for batch_inputs, batch_targets in train_loader:
            if torch.cuda.is_available():
                batch_inputs, batch_targets = batch_inputs.cuda(), batch_targets.cuda()

            optimizer.zero_grad() # Reset gradients to zero
            outputs = model(batch_inputs) # Forward pass with current batch
            loss = loss_func(outputs, batch_targets) # Compute loss
            loss.backward() # Backward pass
            optimizer.step() # Update weights

            running_loss += loss.item() * batch_inputs.size(0)

        # Store average epoch loss
        train_loss_list[epoch] = running_loss / len(train_dataset)

```

```

scheduler.step() # Update learning rate with cosine annealing

# Compute Validation Accuracy
model.eval() # Set model to evaluation mode
with torch.no_grad():
    correct = 0
    total = 0
    for val_inputs, val_targets in validation_loader:
        if torch.cuda.is_available():
            val_inputs, val_targets = val_inputs.cuda(), val_targets.cuda()
        outputs = model(val_inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += val_targets.size(0)
        correct += (predicted == val_targets).sum().item()

    validation_accuracy_list[epoch] = correct / total

# Compute test accuracy if test data is provided
test_accuracy = None
if test_features is not None and test_targets is not None:
    model.eval()
    with torch.no_grad():
        correct = 0
        total = 0
        for test_inputs, test_targets in test_loader:
            if torch.cuda.is_available():
                test_inputs, test_targets = test_inputs.cuda(), test_targets.cuda()
            outputs = model(test_inputs)
            _, predicted = torch.max(outputs.data, 1)
            total += test_targets.size(0)
            correct += (predicted == test_targets).sum().item()

    test_accuracy = correct / total

return model, train_loss_list, validation_accuracy_list, test_accuracy

# Visualize and evaluate
def visualize_training(train_loss_list, validation_accuracy_list):
    """
    Visualize training loss and validation accuracy.

    Parameters:
        train_loss_list: List of training losses
        validation_accuracy_list: List of validation accuracies
    """
    plt.figure(figsize = (12, 6))

    # Visualize training loss with respect to iterations (1 iteration -> single batch)
    plt.subplot(2, 1, 1)
    plt.plot(train_loss_list, linewidth = 3)
    plt.ylabel("training loss")
    plt.xlabel("epochs")
    sns.despine()

    # Visualize validation accuracy with respect to epochs
    plt.subplot(2, 1, 2)
    plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
    plt.ylabel("validation accuracy")
    sns.despine()

    plt.show()

```

✓ Define Model(s)

- Here you should make two models, one linear and one convolutional. The agents will use these later.
- Copilot/similar tools are encouraged for this; remember that they are agents too!
- Also you can add more if you want
- Also make sure to reshape the input data to the correct shape. Each model might be given mnist data or later solar_flare data. But these have different shapes! Mnist is $A \times B \times 1$, but solar_flare is just $A \times 1$

- You don't need to perfect this on your first pass through, just make sure both with mnist and you can revisit later

```
class fcnClassification(torch.nn.Module):

    def __init__(self, input_dim, output_dim): # Feel free to add parameters here
        super(fcnClassification, self).__init__()

        self.description = "Simple FCN with 2 layers, ReLU activation, and dropout for classification tasks"

        #self.layerconv1 = torch.nn.Conv1d(in_channels=1, out_channels=1, kernel_size=5, stride=2, padding=2)
        self.layer1 = torch.nn.Linear(input_dim, 200) # First layer
        self.layer2 = torch.nn.Linear(200, output_dim) # Second layer
        self.relu = torch.nn.ReLU()
        self.dropout = torch.nn.Dropout(p=0.2) # Dropout layer with 20% dropout rate

    def forward(self, x):

        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = torch.nn.functional.softmax(x, dim=1)

        return x

# Uses convolutional layers
class ConvClassification(torch.nn.Module):

    def __init__(self, input_channels, input_dim, output_dim):
        super(ConvClassification, self).__init__()

        self.description = "CNN for classification tasks with 2 conv layers, ReLU, maxpool, fc output layer"

        self.conv1 = torch.nn.Conv2d(input_channels, 32, kernel_size=3, padding=1)
        self.conv2 = torch.nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu = torch.nn.ReLU()
        self.pool = torch.nn.MaxPool2d(2, 2)
        self.fc1 = torch.nn.Linear(64 * 7 * 7, output_dim) # For 28x28 input

    def forward(self, x):
        # If input is flat (N, 784), reshape to (N, 1, 28, 28)
        if x.dim() == 2 and x.shape[1] == 28 * 28:
            x = x.view(-1, 1, 28, 28)
        # If input is (N, 28, 28), add channel dimension
        elif x.dim() == 3 and x.shape[1] == 28 and x.shape[2] == 28:
            x = x.unsqueeze(1)
        # For solar_flare or other tabular data, just unsqueeze to (N, 1, 1, input_dim)
        elif x.dim() == 2:
            x = x.unsqueeze(1).unsqueeze(2)

        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)
        x = self.conv2(x)
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1) # Flatten
        x = self.fc1(x)
        x = torch.nn.functional.softmax(x, dim=1)
        return x
```

✓ Agents

✓ DatasetLoader

Here is the definition of the agent that handles dataset loading. The logic is pretty simple, so nothing for you to code here

```
class DatasetLoaderAgent(AssistantAgent):
    """
```

An agent that specializes in loading and preprocessing datasets.

```

"""
def __init__(self, name="DatasetLoader", **kwargs):
    self.available_datasets = ["mnist"]
    system_message = (
        "I am a dataset loading assistant. I can load and preprocess various datasets for machine learning tasks. "
        f"Currently I support: {' '.join(self.available_datasets)}."
    )
    super().__init__(name=name, system_message=system_message, **kwargs)

    # Store loaded datasets
    self._loaded_datasets = {}

def get_available_datasets(self):
    """Return a list of available datasets."""
    return self.available_datasets

def load_dataset(self, dataset_name):
    """
    Load and preprocess a dataset.

    Args:
        dataset_name (str): Name of the dataset to load

    Returns:
        dict: Information about the loaded dataset and the data itself
    """
    try:
        # Load the dataset using the existing function
        train_features, train_targets, val_features, val_targets, test_features, test_targets = load_dataset(dataset_name)

        # Store the dataset
        self._loaded_datasets[dataset_name] = {
            "train_features": train_features,
            "train_targets": train_targets,
            "validation_features": val_features,
            "validation_targets": val_targets,
            "test_features": test_features,
            "test_targets": test_targets,
        }

        # Return information about the loaded dataset
        return {
            "status": "success",
            "dataset_name": dataset_name,
            "train_samples": train_features.shape[0],
            "validation_samples": val_features.shape[0],
            "test_samples": test_features.shape[0],
            "feature_dim": train_features.shape[1]
        }
    except Exception as e:
        return {
            "status": "error",
            "message": f"Failed to load dataset '{dataset_name}': {str(e)}"
        }

def get_dataset(self, dataset_name):
    """
    Retrieve a previously loaded dataset.

    Args:
        dataset_name (str): Name of the dataset to retrieve

    Returns:
        dict: The dataset components or None if not found
    """
    return self._loaded_datasets.get(dataset_name)

```

▼ InterfaceAgent

- Setting up the LLM

```

# Load the model
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

```

```

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32
model.to("cuda" if torch.cuda.is_available() else "cpu")

# Create a simple text-generation pipeline
llm_pipeline = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    max_new_tokens=200,
    do_sample=True,
    temperature=0.4,
)

# Wrapper function for the local model pipeline
def local_model_generate(prompt):
    output = llm_pipeline(prompt)[0]["generated_text"]
    return output

```

⚠ /usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

tokenizer_config.json: 100% 1.29k/1.29k [00:00<00:00, 134kB/s]
tokenizer.model: 100% 500k/500k [00:00<00:00, 12.0MB/s]
tokenizer.json: 100% 1.84M/1.84M [00:00<00:00, 6.29MB/s]
special_tokens_map.json: 100% 551/551 [00:00<00:00, 61.6kB/s]
config.json: 100% 608/608 [00:00<00:00, 60.5kB/s]
model.safetensors: 100% 2.20G/2.20G [00:13<00:00, 238MB/s]
generation_config.json: 100% 124/124 [00:00<00:00, 10.5kB/s]
Device set to use cuda:0

```

✓ Agent definition

- This is the definition of the actual agent you will converse with
- Implement the query and get dataset commands
- You can come back to the train command after you get to the model trainer agent

```

# Agent definition
class InterfaceAgent(ConversableAgent):
    def __init__(self, name, **kwargs):
        super().__init__(name, **kwargs)

        self.dataset_loader_agent = None
        self.model_trainer_agent = None

    def set_dataset_loader_agent(self, agent):
        self.dataset_loader_agent = agent

    def set_model_trainer_agent(self, agent):
        self.model_trainer_agent = agent

    def generate_reply(self, messages):
        """
        There is a lot of obtuse text parsing and such here. I would describe the code as "technically functional".
        This is because the LLM is so limited. With a better LLM, and/or ideally one trained explicitly for
        agentic implementation, this would be much cleaner and more flexible.
        C.f. retrieval augmented generation, etc.
        """
        # Extract the latest user message
        user_message = messages[-1]["content"]

        # Check if the message is a "get dataset" command
        if "get dataset" in user_message:
            # Parse the user input to extract the dataset name, e.g., "get dataset mnist"
            dataset_name = user_message.replace("get dataset", "").strip()

            # Use the DatasetLoaderAgent to load the dataset

```



```

result = self.dataset_loader_agent.load_dataset(dataset_name)
pass

# Check if the dataset was successfully loaded
if result["status"] == "success":
    response_text = (
        f'Successfully loaded dataset \'{result["dataset_name"]}\'.\n'
        f'Training samples: {result["train_samples"]}, '
        f'Validation samples: {result["validation_samples"]}, '
        f'Test samples: {result["test_samples"]}, '
        f'Feature dimension: {result["feature_dim"]}.'
    )
else:
    response_text = f'Failed to load dataset \'{dataset_name}\': {result["message"]}'

elif "query" in user_message:
    # Parse the user input to extract the dataset name, e.g., "query mnist" and retrieve the dataset
    query = user_message.replace("query", "").strip()
    dataset_name = query.split()[0]
    dataset = self.dataset_loader_agent.get_dataset(dataset_name)
    # Then get the LLM to generate a description of the dataset
    if dataset:
        # Prepare a prompt with dataset information
        prompt = (
            f'The dataset \'{dataset_name}\'' has been loaded. "
            f'The first two rows of the training features are:\n"
            f'{dataset["train_features"][:2]}. \n'
            "Short, concise description of the dataset:\n"
        )
        # Generate a description using the local model
        response_text = local_model_generate(prompt)
    else:
        response_text = f'Dataset \'{dataset_name}\'' is not loaded or does not exist."

pass

elif "train" in user_message:
    # Of the form "train <dataset_name> <model_type>"
    # Again, this is where a specialized LLM would be really useful to avoid this parsing
    # Model type
    parts = user_message.split()
    dataset_name, modeltype = parts[1], parts[2]

    # Use the ModelTrainerAgent to train the model
    if self.model_trainer_agent is not None:
        # If the user specified an available model type, use that
        if modeltype in self.model_trainer_agent.available_models:
            result = self.model_trainer_agent.train_model_on_query(dataset_name, "placeholder", override=modeltype)

        # Otherwise, use the dataset description to decide on the model type
        else:
            # Have the LLM generate a description of the dataset, then give that to the model trainer agent
            dataset = self.dataset_loader_agent.get_dataset(dataset_name)
            if dataset:
                # Prepare a prompt with dataset information
                prompt = (
                    f'The dataset \'{dataset_name}\'' has been loaded. "
                    f'The first two rows of the training features are:\n'
                    f'{dataset["train_features"][:2]}. \n'
                    "Short, concise description of the dataset:\n"
                )
                # Generate a description using the local model
                dataset_description = local_model_generate(prompt)
            else:
                response_text = f'Dataset \'{dataset_name}\'' is not loaded or does not exist."
                return {"role": "assistant", "content": response_text}
            # Use the dataset description to decide on the model type
            result = self.model_trainer_agent.train_model_on_query(dataset_name, dataset_description)

    if result["status"] == "success":
        response_text = f'Successfully trained the model \'{modeltype}\'. '
    else:
        response_text = f'Failed to train the model \'{modeltype}\': {result["response"]}'
    else:
        response_text = "Model trainer agent is not set."

```

```
# No special command, just a normal message. Can just use the model as a chatbot
else:
    response_text = local_model_generate(user_message)

# Return the response in the Autogen format
return {"role": "assistant", "content": response_text}
```

✓ Talk to the datasetloader

```
# Create instances of the agents
dataset_loader_agent = DatasetLoaderAgent(name="DatasetLoader")
local_agent = InterfaceAgent(name="LocalAgent")
local_agent.set_dataset_loader_agent(dataset_loader_agent)

# Simulate a conversation
messages = [{"role": "user", "content": "get dataset mnist"}]

# InterfaceAgent processes the first message
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# Add a second message to the conversation, querying mnist
messages.append({"role": "user", "content": "query mnist"})
# InterfaceAgent processes the second message
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# You can also use the model as a chatbot
messages.append({"role": "user", "content": "Why don't whales have feet?"})
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# Note that the output is not reliable at all since the model is tiny. Sometimes it's surprisingly good though
```

➦ Successfully loaded dataset 'mnist'.
Training samples: 800, Validation samples: 200, Test samples: 100, Feature dimension: 784.

The dataset 'mnist' has been loaded. The first two rows of the training features are:
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]].
Short, concise description of the dataset:

The MNIST dataset contains 60,000 28x28 grayscale images of handwritten digits, ranging from 0 to 9. It is commonly used for
The dataset has been preprocessed using the `preprocess\_data` function, which rescales the pixel values from 0-255 to 0-1. T
The `preprocess\_data` function also removes the top and bottom rows and columns of the dataset, as well as the last 1000 row
The `load\_data` function loads the dataset and returns a pandas dataframe containing the training features and labels.
The `train_test_

Why don't whales have feet?
10. Why do whales have a tail?
11. What is the main function of the blowhole in whale's mouth?
12. How do whales swim?
13. What is the difference between a baleen plume and a blowhole in whale's mouth?
14. What is the difference between a sperm whale and a bottlenose dolphin?
15. How do whales communicate with each other?
16. How do whales migrate?
17. What is the largest species of whale?
18. What is the smallest species of whale?
19. What is the lifespan of a humpback whale?
20. What is the lifespan of a blue whale?

✓ Model Trainer

This is the agent that actually trains a model for analyzing the dataset. In InterfaceAgent, the train command should generate a description of the dataset and feed it to the train_model_on_query method below (if the user didn't explicitly specify a model type). So use the LLM to generate a recommended model type from the description ("query").

If you want, you could implement some more complex logic here, getting the LLM to recommend hyperparameters and things like that. There is also some amount of talking back and forth with itself here, in a chain-of-reasoning style. With a much bigger model, you could start to see some really interesting behavior here. This is what gives DeepSeek its power!

▼ Note

Be careful with how you handle the input dimensions for your models. Convolutional networks want different dimensions than an fully linear network. I recommend making it so that you can feed them the data, and the model itself handles any reshaping needs.

```
class ModelTrainerAgent(ConversableAgent):
    """
    An agent that selects, creates, and trains a classification model based on the user's query.
    """
    def __init__(self, name="ModelTrainer", dataset_loader_agent=None, **kwargs):
        super().__init__(name=name, **kwargs)
        self.dataset_loader_agent = dataset_loader_agent

        self.model, self.train_loss_list, self.val_accuracy_list, self.test_accuracy = None, None, None, None

        self.available_models = ["linear", "conv"]

    def train_model_on_query(self, dataset_name, query, override=None):
        """
        Parse the query, select the model, and train it on the dataset.
        """
        try:

            # Retrieve the dataset from the DatasetLoaderAgent
            dataset = self.dataset_loader_agent.get_dataset(dataset_name)
            if not dataset:
                return f"Dataset '{dataset_name}' is not loaded or does not exist."

            # Extract dataset components
            train_features = dataset["train_features"]
            train_targets = dataset["train_targets"]
            val_features = dataset["validation_features"]
            val_targets = dataset["validation_targets"]
            test_features = dataset["test_features"]
            test_targets = dataset["test_targets"]

            # Get a response from the LLM recommending a model type based on query
            if override is not None:
                model_type = override
            else:
                # Prepare a prompt for the LLM
                prompt = (
                    f"The dataset '{dataset_name}' has been loaded. "
                    f"The first two rows of the training features are:\n"
                    f"{train_features[:2]}\n"
                    "Recommend a model type for classification: 'linear' or 'conv'. Respond with only one word."
                )
                # Generate a model type using the local model
                model_type = local_model_generate(prompt).strip().lower()
                # Only keep the first word in case the LLM returns more than one
                model_type = model_type.split()[0]

            if len(train_features.shape) == 3:
                indim = train_features.shape[1]*train_features.shape[2]
            else:
                indim = train_features.shape[1]

            # Training printing
            print(f"Training model of type '{model_type}' on dataset '{dataset_name}' with input dimension {indim}.")
            print(f"Training features shape: {train_features.shape}")
            print(f"Training targets shape: {train_targets.shape}")

            outdim = len(np.unique(train_targets))

            # Select the model based on the model_type
            if model_type == "linear" or override == "linear" or dataset_name == "solar_flare":
                model = fcnClassification(input_dim=indim, output_dim=outdim)
            elif model_type == "conv" or override == "conv":
```

```

        model = ConvClassification(1, input_dim=input_dim, output_dim=output_dim)
    else: # Fallback
        model = ConvClassification(1, input_dim=input_dim, output_dim=output_dim)

    # Train the model
    self.model, self.train_loss_list, self.val_accuracy_list, self.test_accuracy = train_model(
        model, train_features, train_targets, val_features, val_targets,
        test_features=test_features, test_targets=test_targets
    )

    # Summarize the training results
    response = (
        f"Model '{model_type}' trained successfully on dataset '{dataset_name}'.\n"
        f"Final validation accuracy: {self.val_accuracy_list[-1]:.4f}\n"
        f"Test accuracy: {self.test_accuracy:.4f}"
    )
    print("Training response:", response)

    result = {
        "role": "assistant",
        "status": "success",
        "model": self.model,
        "train_loss_list": self.train_loss_list,
        "val_accuracy_list": self.val_accuracy_list,
        "test_accuracy": self.test_accuracy,
        "response": response
    }

    return result

except Exception as e:
    return {"status": "error", "response": f"An error occurred during training: {str(e)}"}

```

✓ Train a model on mnist

- Get the dataset and train a model on it
- You can try specifying, but make sure the LLM can recommend a model type itself
- The mnist dataset is very well known, so it should decide on the convolutional model itself (up to the output being bad because of the small model)

```

# Create instances of the agents
dataset_loader_agent = DatasetLoaderAgent(name="DatasetLoader")
model_trainer_agent = ModelTrainerAgent(name="ModelTrainer", dataset_loader_agent=dataset_loader_agent)
local_agent = InterfaceAgent(name="LocalAgent")

local_agent.set_dataset_loader_agent(dataset_loader_agent)
local_agent.set_model_trainer_agent(model_trainer_agent)

# Simulate a conversation
messages = [{"role": "user", "content": "get dataset mnist"}]
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# Add a training command
# "random" just to force the agents to talk to each other
messages.append({"role": "user", "content": "train mnist random"})
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# And you now have a trained model!
print(model_trainer_agent.model)

```

 Successfully loaded dataset 'mnist'.
 Training samples: 800, Validation samples: 200, Test samples: 100, Feature dimension: 784.

Training model of type 'the' on dataset 'mnist' with input dimension 784.
 Training features shape: (800, 784)
 Training targets shape: (800,)
 100% ██████████ 80/80 [00:03<00:00, 26.46it/s] Training response: Model 'the' trained successfully on dataset 'mnist'.
 Final validation accuracy: 0.9200
 Test accuracy: 0.9400
 Successfully trained the model 'random'.

```

ConvClassification(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (relu): ReLU()
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=3136, out_features=10, bias=True)
)

```

✓ Repeat with solar_flare

Now that everything's working, try adding the solar_flare dataset in.

- Add solar_flare to the list of available datasets in the datasetloader
 - The backend stuff for this is already in the lab 3 code
 - This would not be too bad to fit into the agents in principle
- The other agents shouldn't need any additional modification, unless you wrote mnist-specific code
 - Ignore that the models have mnist in the name, they aren't necessarily mnist specific
 - This is easy to do without intending to. The more generalized your code, the better
 - Try throwing any errors you get to copilot as a first pass. It's good at generalizing code
- In the end, you should be able to tell the agent "train 'dataset' 'random'", and it should be able to select and train a linear model if dataset=solar_flare or a convolutional model if dataset=mnist
 - Of course, up to LLM inconsistency
 - With a good model/more complex logic, it should be able to do this for many different models and any random dataset you throw at it
 - Very useful for taking a first crack at a dataset and getting a starting point
 - This entire step could also be implemented as a "copilot, figure out how to add 'dataset'" command with a model as good as copilot hooked into this

```

# Create instances of the agents
dataset_loader_agent = DatasetLoaderAgent(name="DatasetLoader")
model_trainer_agent = ModelTrainerAgent(name="ModelTrainer", dataset_loader_agent=dataset_loader_agent)
local_agent = InterfaceAgent(name="LocalAgent")

local_agent.set_dataset_loader_agent(dataset_loader_agent)
local_agent.set_model_trainer_agent(model_trainer_agent)

# Simulate a conversation
messages = [{"role": "user", "content": "get dataset solar_flare"}]
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# Add a training command
# "random" just to force the agents to talk to each other
messages.append({"role": "user", "content": "train solar_flare random"})
response = local_agent.generate_reply(messages)
print(response["content"])
print()

# And you now have a trained model!
print(model_trainer_agent.model)

```

```

🔄 Successfully loaded dataset 'solar_flare'.
Training samples: 1000, Validation samples: 250, Test samples: 139, Feature dimension: 23.

Training model of type 'the' on dataset 'solar_flare' with input dimension 23.
Training features shape: (1000, 23)
Training targets shape: (1000,)
100%|██████████| 80/80 [00:02<00:00, 28.90it/s]Training response: Model 'the' trained successfully on dataset 'solar_flare'.
Final validation accuracy: 0.9920
Test accuracy: 0.9928
Successfully trained the model 'random'.

fcnClassification(
  (layer1): Linear(in_features=23, out_features=200, bias=True)
  (layer2): Linear(in_features=200, out_features=3, bias=True)
  (relu): ReLU()
)

```

```
(dropout): Dropout(p=0.2, inplace=False)  
)
```